

Git and GitHub

Complete Classroom Material

**Prepared by Euron
Sudhanshu Kumar**

1. Why Git is Needed

When multiple developers work on the same software project, they need a reliable way to track changes, work independently, combine their work, review history and recover from mistakes. Git provides this version-control layer.

Without version control, teams often create many manual copies of files and lose clarity about which version is correct. Git replaces that approach with a structured history of commits.

2. What is Git?

Git is a distributed version control system. It records changes to project files over time. Each developer can have a complete local repository containing the project and its history.

A commit is a recorded point in that history. Developers can inspect earlier commits, compare versions, create independent branches and combine changes later.

3. What is GitHub?

GitHub is a cloud platform built around Git repositories. Git handles version control; GitHub adds remote hosting and collaboration features such as Pull Requests, code reviews, Issues, repository permissions and GitHub Actions.

Git does not require GitHub. A Git repository can also be hosted on other platforms or used entirely locally.

4. Core Git Terminology

Repository: A project tracked by Git.

Commit: A recorded point in the repository history.

Branch: An independent line of development.

Remote: A named reference to another Git repository.

Origin: The conventional default name of the primary remote.

HEAD: A reference to the currently checked-out position.

Clone: Creating a local copy of an existing repository.

Fork: Creating your own server-side copy of another GitHub repository.

Pull Request: A request on GitHub to review and merge changes from one branch into another.

5. Git Working Model

The most important concept to understand is how a change moves through Git.

```
Working Directory
|
| git add
v
```

```
Staging Area
  |
  | git commit
  v
Local Repository
  |
  | git push
  v
Remote Repository
```

A simple way to remember the flow is: write the code, select the changes, record the changes, and then synchronize them with the remote repository.

6. Installing and Configuring Git

Check whether Git is installed:

```
git --version
```

Configure the name and email that should appear in commits:

```
git config --global user.name "Sudhanshu Kumar"
git config --global user.email "your-email@example.com"
git config --list
```

7. Creating the First Repository

```
mkdir git-demo
cd git-demo
git init
git status
```

git init creates a Git repository in the current directory. Git stores repository metadata in a hidden .git directory. git status should be used frequently because it shows the current state of the working directory and staging area.

8. Tracking and Committing Files

Create app.py and add:

```
print("Welcome to Euron")
```

Check the repository, stage the file and create the first commit:

```
git status
git add app.py
git commit -m "Initial commit"
```

To stage current changes across the working tree:

```
git add .
```

9. Viewing Changes and History

```
git diff
git log
```

```
git log --oneline
git log --oneline --graph --decorate --all
```

git diff is useful before staging because it shows unstaged modifications. git log displays commit history. The graph form is especially useful for understanding branches and merges.

10. Using .gitignore

Projects often contain files that should not be tracked, including local environments, generated files and secrets.

```
.env
venv/
__pycache__/
*.pyc
node_modules/
.idea/
.vscode/
```

Passwords, API keys, private keys and database credentials should not be intentionally committed. If a secret has already been committed, simply adding it to .gitignore does not erase the existing history. The exposed credential should be rotated or revoked.

11. Connecting Git to GitHub

After creating an empty repository on GitHub, connect the local repository:

```
git remote add origin <repository-url>
git remote -v
git branch -M main
git push -u origin main
```

origin is simply the conventional name for the remote. It does not mean GitHub itself.

12. Cloning an Existing Repository

```
git clone <repository-url>
cd <repository-folder>
```

Clone is normally used when a repository already exists and a developer wants a local working copy.

13. Fetch and Pull

Fetch downloads information and objects from the remote without automatically integrating them into the current branch.

```
git fetch
```

Pull fetches remote changes and then integrates the upstream branch into the current branch. The integration strategy can use merge or rebase depending on configuration and options.

```
git pull
```

14. Branching

Branches allow developers to build features or fixes without directly changing the main development line.

```
git branch
git switch -c feature/login
```

After making changes on the branch:

```
git add .
git commit -m "Add login feature"
git push -u origin feature/login
```

15. Merging Branches

When the feature is ready, it can be integrated into another branch.

```
git switch main
git merge feature/login
```

A merge may be a fast-forward or may create a merge commit when histories have diverged.

16. Merge Conflicts

A conflict occurs when Git cannot safely decide how competing changes should be combined. This commonly happens when different branches modify the same lines.

```
<<<<<< HEAD
message = "Welcome to Euron"
=====
message = "Welcome to Super30"
>>>>>> feature
```

The developer manually chooses the intended final content, removes the conflict markers, stages the resolved file and completes the merge.

```
message = "Welcome to Euron Super30"

git add app.py
git commit
```

17. Merge and Rebase

Merge combines development histories. Rebase takes commits from one line of development and replays them on top of another base.

```
git switch feature/login
git rebase main
```

Rebase changes commit identities and rewrites history. Developers should be careful when rebasing commits that have already been shared with other team members.

18. Restoring and Reverting Changes

Discard an unstaged modification to a tracked file:

```
git restore app.py
```

Remove a file from the staging area without deleting the working change:

```
git restore --staged app.py
```

Reverse an earlier commit by creating a new commit:

```
git revert <commit-id>
```

Revert is often appropriate for shared branches because it preserves the existing history.

19. HEAD and Reset

HEAD points to the currently checked-out position. HEAD~1 refers to its first parent, HEAD~2 moves two first-parent steps back, and so on.

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

- soft moves HEAD while keeping the changes staged.
- mixed moves HEAD while keeping the changes in the working directory but unstaged.
- hard resets tracked working files and the index to the selected commit and can destroy uncommitted work.

20. Git Stash

Stash temporarily stores unfinished work when a developer needs to change branches or handle another task.

```
git stash
git stash list
git stash apply
git stash pop
git stash drop
```

git stash apply restores a selected stash while normally keeping it in the stash list. git stash pop applies a stash and removes that stash entry when the application succeeds.

21. Git Tags

Tags are commonly used to identify releases or other important commits.

```
git tag v1.0.0
git tag -a v1.0.0 -m "First production release"
git push origin v1.0.0
```

22. Pull Requests and Code Review

A professional workflow normally pushes a feature branch to GitHub and then opens a Pull Request.

```
Feature Branch
|
v
```

```
Pull Request
  |
  v
Automated Checks
  |
  v
Code Review
  |
  v
Approval
  |
  v
Merge into main
```

A Pull Request is different from the git pull command. Pull Requests are a collaboration and review feature on platforms such as GitHub.

23. Fork and Clone

Clone copies a repository to the local machine. Fork creates a separate copy of another repository under your own GitHub account. Forking is common in open-source development.

```
Original Repository
  |
  | Fork
  v
Your GitHub Repository
  |
  | Clone
  v
Local Development
  |
  | Push
  v
Your Fork
  |
  | Pull Request
  v
Original Repository
```

24. GitHub Issues

Issues can be used to track bugs, features and engineering tasks. A mature workflow connects requirements to code changes.

```
Issue
  |
  v
Branch
  |
  v
Development
  |
  v
Commit
```

```
|  
v  
Pull Request  
|  
v  
Review and Merge  
|  
v  
Issue Closed
```

25. GitHub Actions and CI/CD

GitHub Actions can execute automated workflows when code is pushed or a Pull Request is opened. Teams commonly use these workflows for testing, linting, building and deployment.

```
name: Python Tests  
  
on:  
  push:  
    branches: [main]  
  pull_request:  
    branches: [main]  
  
jobs:  
  test:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - uses: actions/setup-python@v5  
        with:  
          python-version: "3.12"  
      - run: pip install -r requirements.txt  
      - run: pytest
```

26. GitHub Authentication

Repositories can commonly be accessed using HTTPS or SSH. GitHub does not accept a normal account password for Git operations over HTTPS. Developers should use a currently supported authentication method such as Git Credential Manager, a personal access token where appropriate, or SSH.

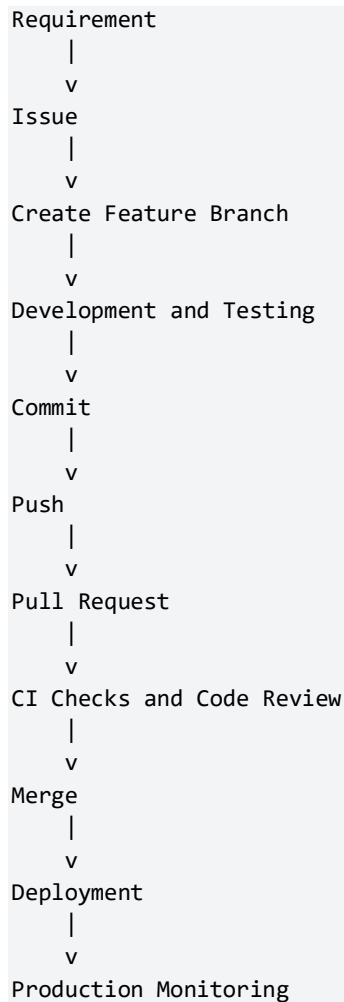
```
ssh-keygen -t ed25519 -C "your-email@example.com"  
ssh -T git@github.com
```

27. Writing Good Commit Messages

Commit messages should explain the purpose of the change. Avoid messages such as changes, final or working now.

```
git commit -m "Add JWT authentication"  
git commit -m "Fix duplicate user registration"  
git commit -m "Add Qdrant retrieval service"  
git commit -m "Add rate limiting middleware"
```

28. Real-World Development Workflow



This is the connection students should understand: Git is not only a set of commands. It is part of the complete software engineering lifecycle.

29. Classroom Practical

Create a small Student Management project and use it to demonstrate the complete workflow.

```
student-api/  
  app.py  
  students.py  
  requirements.txt  
  README.md  
  .gitignore  
  
def get_students():  
    return [  
        {"id": 1, "name": "Aman"},  
        {"id": 2, "name": "Priya"}  
    ]
```

Initialize the repository, make an initial commit, create a feature branch, add the student functionality, push the branch, open a Pull Request, review and merge it. Afterward, deliberately create conflicting changes on two branches and demonstrate conflict resolution.

30. Student Practice Tasks

1. Create a repository with at least three files and make three meaningful commits.
2. Create two feature branches and implement different changes in each.
3. Merge a feature branch into main.
4. Create and resolve a merge conflict intentionally.
5. Use git diff before staging a modification.
6. Stage a file and then unstage it without losing the change.
7. Create multiple stashes and practice stash list, apply and pop.
8. Create a commit and reverse it with git revert.
9. Create a GitHub Issue and develop the fix on a dedicated branch.
10. Open a Pull Request and perform a basic code review.
11. Create and push an annotated release tag.
12. Add a basic GitHub Actions workflow that runs tests.

31. Important Interview Questions

1. What is the difference between Git and GitHub?
2. What is the staging area?
3. What is the difference between git fetch and git pull?
4. What is the difference between merge and rebase?
5. What is a merge conflict and how is it resolved?
6. What is the difference between git stash apply and git stash pop?
7. What is the difference between git revert and git reset?
8. What is HEAD?
9. What is origin?
10. What is the difference between fork and clone?
11. Why should secrets never be committed?
12. What is a Pull Request?
13. How would you safely undo a bad commit on a shared branch?
14. How does Git fit into a CI/CD workflow?

32. Final Revision

```
Write Code
  |
  v
git status and git diff
  |
  v
git add
  |
  v
Staging Area
  |
  v
git commit
  |
  v
Local Repository
  |
  v
git push
  |
  v
Remote Branch
  |
  v
Pull Request
  |
  v
Review and CI
  |
  v
Merge
```

The objective is to understand the workflow rather than memorize commands. Once the workflow is clear, Git becomes much easier to use in real projects.